

本地相似图片检索 MVP 调研与实现建议

ChatGPT

2026-06-04

0. 结论

建议第一版做一个非常小的本地服务：**CLIP 图片/文本 embedding + NumPy 暴力余弦检索 + SQLite FTS5 元信息检索 + pHash 近重复增强 + FastAPI API**。这个组合足够在 MacBook 上快速复现，不需要租显卡，也不需要搭向量数据库。

第一版不要追最新模型，也不要先接 FAISS/Qdrant/Milvus。先把“一个文件夹里的图片能被以图搜图、以文搜图、按元信息补充召回，并能被 Agent 通过 API 调用”跑通。

默认模型建议用 openai/clip-vit-base-patch32。理由是生态稳定、示例多、Hugging Face Transformers 支持成熟，MacBook 上可用 CPU 或 MPS 跑。后续如果需要更好的中文/多语言文本检索，可尝试 SigLIP2；如果更关心本地速度，可尝试 Apple MobileCLIP/MobileCLIP2；如果是文档页面检索，可单独评估 ColPali/ColQwen2，但不建议进入 MVP。

1. 需求边界

本次只做一个聚焦模块：

输入：本地图片文件夹，外加可选 sidecar 元信息文件。

输出：相似图片列表，包含路径、分数、视觉分数、元信息分数、pHash 分数。

查询方式：以图搜图、以文搜图、图片 + 元信息混合检索。

API：本地 FastAPI 服务，方便后续 Agent 调用。

运行环境：MacBook 本地，优先 MPS，自动 fallback CPU。

不做：模型训练、前端、权限系统、云向量数据库、GPU 租赁、大规模分布式索引。

2. 最新进展简要梳理

CLIP / OpenCLIP：CLIP 仍是图文共同 embedding 与零样本检索的经典基线；Hugging Face CLIP 文档明确支持 image/text features 与相似度计算。[R1] MVP 默认用 CLIP，先保证可复现。

SigLIP / SigLIP2: SigLIP2 在 2025 年发布，强调多语言视觉语言编码，官方博客称其在 zero-shot classification、image-text retrieval 等能力上优于旧 SigLIP。[R2] 第二阶段可尝试，尤其是中文/多语言文本查询。

MobileCLIP / MobileCLIP2: Apple 的 MobileCLIP 面向低延迟本地/移动端图文模型；MobileCLIP2 继续强调 3-15ms 低延迟和 50-150M 参数量级。[R3] 它很适合 Mac/端侧，但集成链路比标准 CLIP 多一点，MVP 先做可替换 adapter。

DINOv2: Meta DINOv2 是自监督视觉特征模型，适合图像级视觉任务和 image-to-image 相似性。[R4] 如果后续发现 CLIP 对“同一物体/局部视觉相似”不够敏感，可加 DINOv2 分支。

ColPali / ColQwen2: ColPali 把文档页当图片编码，并用 late interaction 做视觉文档检索，针对 PDF、表格、版式复杂页面有明显价值。[R5] 它对发票/专利/合同很相关，但模型重、实现复杂，MVP 先不用。

元信息/全文检索: SQLite FTS5 是内置全文检索模块，支持在本地 SQLite 中检索文本集合。[R6] 第一版用它存文件名、目录、EXIF、sidecar、OCR 文本，足够轻。

Mac 本地加速: PyTorch MPS 后端可利用 Apple Metal 在 Mac 上跑张量计算。[R7] 代码优先尝试 mps，失败自动回 CPU。

3. 为什么单靠语义图片检索不够

硬盘、显卡、CPU、机器人这类自然图片，CLIP embedding 基本可以直接给出不错的相似结果。但发票、专利、合同、报告截图不一样：它们的模板、排版、表格结构非常接近，视觉 embedding 往往会把同模板图片排在一起，却不能稳定区分供应商、发票号、专利号、日期、金额、标题等关键字段。

所以第一版应该做混合检索，而不是只做 CLIP：

视觉分数: CLIP image embedding 的余弦相似度。

元信息分数: 文件名、目录名、EXIF、sidecar JSON/TXT、可选 OCR 文本的 SQLite FTS5 检索分数。

近重复分数: pHash Hamming distance，用来识别同一张图的压缩版、缩略图或轻微变体。

推荐两个检索 profile：

general：用于硬件、机器人、产品、普通照片。建议权重为 visual 0.85, metadata 0.10, phash 0.05。

document：用于发票、专利、合同、表格截图。建议权重为 visual 0.45, metadata 0.45, phash 0.10。

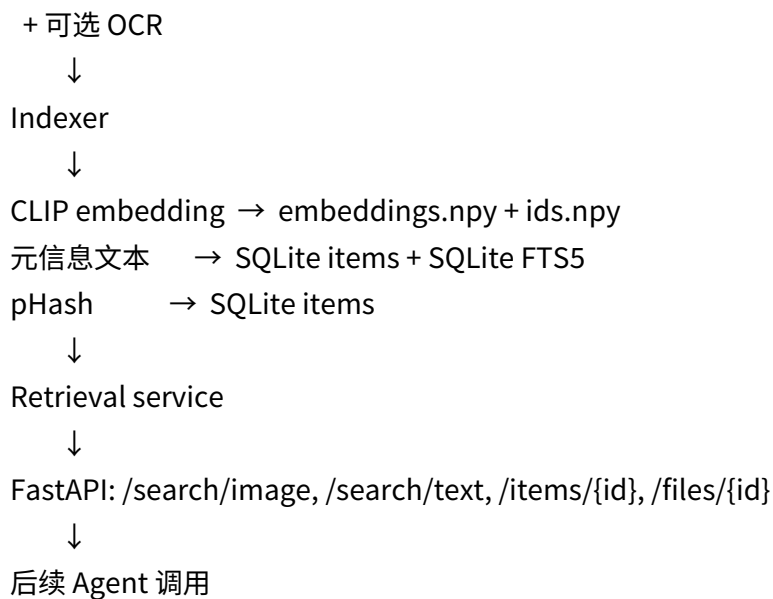
4. MVP 架构

本地图片文件夹

+ 文件系统元信息

+ EXIF

+ sidecar JSON/TXT/MD



这个架构的优势是：

依赖少，便于 Codex 一次实现。

不需要额外服务，不需要 Docker，也不需要显卡。

小数据直接 NumPy 矩阵乘法，排查问题最简单。

元信息和 OCR 文本留在 SQLite，后续要换向量库也不影响 API。

5. 本地快速复现流程

5.1 准备图片

先手工放一个文件夹即可：

```
samples/  
  hard_disk/  
  gpu/  
  cpu/  
  robot/  
  invoice/  
  patent/
```

硬盘、显卡、CPU、机器人图片可以从 Hugging Face、公开数据集或本地图片手工下载。算法本身不依赖数据集格式；Hugging Face Datasets 的 ImageFolder 也支持直接从本地文件夹创建图像数据集。[R8]

文档类图片建议配一个同名 .json 或 .txt：

```
samples/invoice/invoice_001.png
samples/invoice/invoice_001.json
samples/invoice/invoice_001.txt
```

5.2 安装与索引

```
python3 -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -r requirements.txt
```

```
python -m app.cli index \
--image-dir ./samples \
--data-dir ./data \
--model-name openai/clip-vit-base-patch32 \
--batch-size 16 \
--ocr none \
--reset
```

5.3 命令行检索

```
python -m app.cli search-image \
--image ./samples/gpu/001.jpg \
--data-dir ./data \
--top-k 5 \
--profile general
```

文档类：

```
python -m app.cli search-image \
--image ./samples/invoice/query.png \
--data-dir ./data \
--top-k 5 \
--profile document \
--metadata-query "AMD INV-2026-001"
```

5.4 启动 API

FastAPI 是一个基于 Python 类型提示的高性能 Web 框架，适合快速暴露本地模型服务 API。[R9]

```
uvicorn app.api:app --host 127.0.0.1 --port 8000 --reload
```

```
curl -X POST http://127.0.0.1:8000/search/image \
-F "file=@./samples/gpu/001.jpg" \
```

```
-F "top_k=5" \  
-F "profile=general"
```

6. API 设计

GET /health

返回服务状态、模型名、设备、索引图片数量。

```
{  
  "status": "ok",  
  "device": "mps",  
  "model": "openai/clip-vit-base-patch32",  
  "items": 1234  
}
```

POST /search/image

Multipart form:

file: 查询图片。

top_k: 默认 10。

profile: general 或 document。

metadata_query: 可选，用于发票号、专利号、供应商、日期等。

candidate_k: 默认 100。

返回：

```
{  
  "query_type": "image",  
  "top_k": 10,  
  "results": [  
    {  
      "id": 12,  
      "rel_path": "gpu/001.jpg",  
      "score": 0.93,  
      "visual_score": 0.91,  
      "metadata_score": 0.50,  
      "phash_score": 0.00,  
      "width": 1024,  
    }  
  ]  
}
```

```
"height": 768,  
"metadata": {"folder": "gpu"}  
}  
]  
}
```

POST /search/text

JSON body:

```
{  
  "text": "graphics card with three fans",  
  "metadata_query": "gpu",  
  "top_k": 10,  
  "profile": "general"  
}
```

逻辑：用 CLIP text embedding 做图文共同空间检索，同时用 metadata_query 或 text 做 FTS5 检索。

GET /items/{id} 与 GET /files/{id}

分别返回元信息与图片文件。/files/{id} 必须确认真实路径在允许的图片根目录下，避免 path traversal。

7. Codex 实现切分

给 Codex 的 Markdown 已经单独整理成实现说明。建议按下面顺序实现：

metadata.py：扫描文件、EXIF、sidecar、pHash。

embedder.py：CLIP image/text embedding，MPS/CPU fallback。

index_store.py：SQLite + FTS5 + NumPy embeddings 文件。

retrieval.py：视觉检索、FTS 检索、pHash、分数融合。

cli.py：index/search 命令。

api.py：FastAPI endpoints。

tests + README。

8. 验收标准

第一版验收不看模型排行榜，只看能不能快速复现：

20-100 张图片即可索引成功。

用 samples/gpu/001.jpg 搜索，top-5 大部分是显卡或相近硬件。

用机器人图片搜索，top-5 大部分是机器人。

发票/专利模板类图片，如果只靠视觉检索结果混在一起；加 metadata_query 或 sidecar/OCR 后，目标文件排名明显提升。

uvicorn app.api:app 启动后，Agent 能用 HTTP 调用 /search/image 和 /search/text。

没有 MPS、没有 OCR、没有 Hugging Face 样本下载时，核心功能仍可运行。

9. 后续升级路径

图片超过 50k，NumPy 暴力检索变慢：再引入 FAISS、hnswlib 或 LanceDB。FAISS 是成熟的 dense vector similarity search 库。[R10]

中文文本查询效果一般：尝试 SigLIP2 或多语言 CLIP/SigLIP 模型。

对同一物体、局部视觉相似不敏感：增加 DINOv2 embedding 分支。

发票/专利页面强依赖版式和表格：评估 ColPali/ColQwen2 visual document retrieval。

近重复检测不稳：增加 pHash、dHash 和更严格阈值。pHash/imagehash 适合判断图片是否“看起来近似相同”，不同于加密哈希。[R11]

Agent 需要跨机器调用：增加简单 token、只读文件服务和 Dockerfile。

10. 推荐最终技术选型

第一版固定如下：

模型：openai/clip-vit-base-patch32

推理：PyTorch，优先 MPS，fallback CPU

图片处理：Pillow

近重复：imagehash pHash

元信息库：SQLite + FTS5

向量存储：embeddings.npy + ids.npy

API：FastAPI + Uvicorn

OCR：默认关；需要文档场景时再开 Tesseract/RapidOCR

这套方案的核心价值不是“最强”，而是“最快能跑通、最容易 debug、后续可替换”。

References

[R1] Hugging Face Transformers, CLIP documentation.

https://huggingface.co/docs/transformers/en/model_doc/clip

[R2] Hugging Face Blog, “SigLIP 2: A better multilingual vision language encoder” , 2025-02-21.

<https://huggingface.co/blog/siglip2>

[R3] Apple Machine Learning Research, “MobileCLIP2: Improving Multi-Modal Reinforced Training” .

<https://machinelearning.apple.com/research/mobileclip2>

[R4] Meta AI, “DINOv2: State-of-the-art computer vision models with self-supervised learning” , 2023-

04-17. <https://ai.meta.com/blog/dino-v2-computer-vision-self-supervised-learning/>

[R5] arXiv, “ColPali: Efficient Document Retrieval with Vision Language Models” , 2024.

<https://arxiv.org/abs/2407.01449>

[R6] SQLite Documentation, FTS5 Extension. <https://sqlite.org/fts5.html>

[R7] Apple Developer, “Accelerated PyTorch training on Mac” .

<https://developer.apple.com/metal/pytorch/>

[R8] Hugging Face Datasets documentation, “Create an image dataset / ImageFolder” .

https://huggingface.co/docs/datasets/en/image_dataset

[R9] FastAPI documentation. <https://fastapi.tiangolo.com/>

[R10] Faiss documentation. <https://faiss.ai/index.html>

[R11] Johannes Buchner, imagehash GitHub repository.

<https://github.com/JohannesBuchner/imagehash>